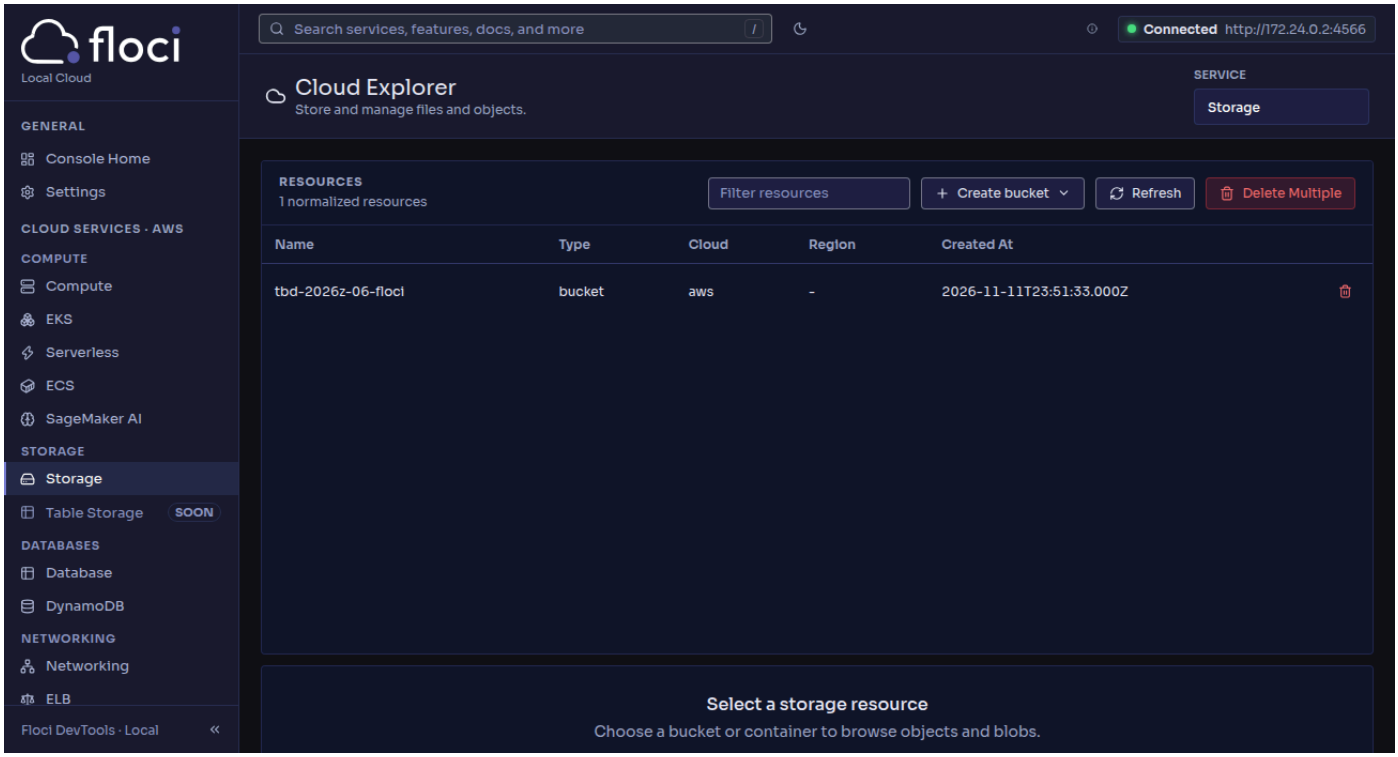
Local cloud emulator (floci): team 06

One S3 bucket, `tbd-2026z-06-floci`, created with the `hashicorp/aws` provider (`~> 6.0`, region `eu-central-1`, no endpoint and no keys in the code) against floci `2.4.0` running in Docker. `AWS_ENDPOINT_URL=http://localhost.floci.io:4566` sent every AWS call to the emulator. `terraform apply` took under a second. Files: `floci/compose.yml`, `floci/main.tf`. Everything was destroyed afterwards (`terraform destroy`, `docker rm -f floci-aws-ui`, `docker compose down`).

Console screenshot: Storage, the bucket



How could an emulator help in this project, and what can't it replace?

What it helps with

- **Cost.** No credits and no billing. In this project every real `mise run up` and every test run spends Snowflake credits (12 credit cap) and GCP coupon money, and a forgotten warehouse or a 1-minute dynamic table lag keeps burning them. The emulator costs nothing and can be recreated as often as needed.
- **Development time.** The bucket was created in under a second; `mise run up` takes about 10 minutes, with Terraform asking for approval at each root, and GCP project IDs cannot be reused for 30 days after a destroy. An emulator allows many quick edit-apply-destroy cycles to debug Terraform code (variables, `for_each`, module wiring, naming limits such as the 30-character project ID) before touching the real cloud.
- **CI.** A container started in the pipeline gives a clean, disposable "cloud" on every pull request: `terraform validate/plan/apply` and integration tests can run without cloud credentials, without secrets in CI and without a bill, and they are deterministic.

What it cannot replace (and I would still test on the real cloud)

- **The platforms this project is actually about.** floci emulates AWS, but this project runs on Google Cloud and Snowflake. Snowflake is a SaaS service with no emulator: Snowpipe auto-ingest, Snowpipe Streaming, dynamic tables, dbt projects, masking and row access policies, tags and the Horizon Iceberg REST catalog (scan planning, vended credentials) exist only in the real service. The same Terraform code cannot be reused either: only the workflow transfers.
- **Identity and permissions across services.** The trust between Snowflake's storage integration / external volume service account and the GCS bucket, IAM propagation delays (the project waits for the grants to propagate before Snowflake can write), PATs, authentication policies. Emulators usually accept any credentials, so a missing grant would pass locally and fail for real.
- **Behaviour, limits and timing.** Real latency (Snowpipe 19 s to 2 min 45 s here), eventual consistency, quotas, service limits, API differences and bugs between the emulator and the real service.
- **Cost and operations.** Real billing and budgets (resource monitor, budget calling `STOP_ALL()`), warehouse and serverless consumption, and security properties such as private access settings.

So an emulator is good for the inner loop (fast, free feedback on the infrastructure code), but the final acceptance of this lakehouse still has to be run once on the real GCP and Snowflake.